Mapping data type :-

→ a mapping dtype is a container data structure that links a unique identifier (a key) to a piece of data (value)
→ This allows you to look up information efficiently without needing know exact numerical index

Dictionaries :-

Why do we need a dict ?

```
names = ['Ravi', 'Krina', 'Sita']
ages = [22, 25, 21]
```

→ Now dict represent realationship directly

```
students = {
    "Ravi": 22,
    "Kiran": 25,
    "Sita": 21
}
```

→ Meaningful key → data.

Dictionary :-

→ Dictionaries are used to store data values in key-value pairs
→ Dictionary is collection which is ordered
→ changeble and do not allow duplicate

→ from python 3.7 is dict are ordred. Python 3.6 dict is unordred.

Creation Dictionaries:

→ Dictionaries are written with curly brackets and have keys and values.

```
d = { "brand": "Ford"
      "model": "mustang"
      "year": 1964
    }

print(d)
```

Dictonary items:

→ Dictonary items are ordered, changeble, and do not allow duplicats

Duplicatcy not allowed:

→ Dictionaries can not have two items with same key.

```
d = {
    "name": "gurunadh"
    "age": 22
    "age": 23
    }

# {"name": 'gurunadh', "age": 23}
```

override first value.

Dictionary length

→ using the length function

```
d = {"name": "guru", "age": 23}
print(len(d))   # 2
```

dict() - Constructor:

-> it take the key word arguments and create a dictionary

Syntax

dict(keywordarguments)

a = dict(name = "guru", age = 36, country = "india")

Access Dictionary items:-

-> you can access the items of dictionary by referring of its key name

d = {'name': 'guru', 'age': 21}

print(d["name"]) # guru.

-> get():

-> it is also same like above, i will take the key and return the value:

a = {"name": "guru", 'age": 21}

print(a.get("name")) # "guru"

Syntax

dict.get(keyname, value)

-> keyname -> if it is key value

-> value -> if key is not present then this value will show.

keys():-

-> key method return a view object, the view object contains the keys of Dictionary. As a list.

-> The view object reflects any changes done in dictionary

→ Syntax dict.keys()

→ No parameter values.

```
a = {"name": "guru", "age": 21}
b = a.keys()
print(b)        # dict_keys(['name', 'age'])
```

→ The view object reflect the changes

```
a = {"name": "guru", "age": 21}
b = a.keys()
a["Rod-no"] = 221
print(b)    # dict_keys(["name", "age", "Rod_no"])
```

values() :-

→ The values method return a view object. The view object contains values of the dictionary, as list

→ The view object any change that happend in Dictionary.

```
a = {"name": "guru", "age": 21}
print(a.values())   # dict_values(['guru', 21])
```

→ the view object reflect changes

```
a = {"name": "guru", "age": 21}
print(a.values())
a["Roll-no"] = 221      # dict_values(['guru', 21, 221])
```

items() :-

→ The item method return the view object. the view object contains the key-value pair of dictonary as tuples in list

→ The view object will reflect any done.

Syntax

```
dict.items()
```

→ NO parameter value

```
a = {"name": "guru", "age": 21}
print(a.items()) # dict_items([("name", "guru"), ("age", 21)])
```

→ changes reflect

```
a = {"name": "guru", "age": 21}
print(a.items())
a["Roll-no"] = 221  # dict_items([( ), ( ), ( )])
```

change dictionary items:-

→ using key

→ By using the specific key in Dict we can change the value

```
a = {"Name": "guru", "age": 21}
a["name"] = "sanu"
print(a)
```

→ update() :-

→ update method insert the specified items to the dictionary

→ The specified item is Dict or iterable that have key-value pairs

```
a = {"name": "guru", "age": 21,}
a.update({"Roll-no": 2215})
print(a)
# {"name": "guru", age: 21, "Roll-no": 221}
```

Adding Dictionary items :-

→ Adding by using key :-

→ Adding an item to dictionary is done by new index key and assigning value to it

```
a = {"name": "guru", "age": 21}
a["roll_no"] = 221
print(a) # {'name': 'guru', "age": 21, "roll_no": 221}
```

→ update() → this also take dict iterable update the dictionary

Remove items :-

pop() :-

→ This method removes the specified items from dictionary
→ The value of removed item is the return value pop()

Syntax

dict.pop(keyname, defaultvalue)

→ default value - specified key not present at that time default value will print

popitem() :-

→ Method removes the item that was last inserted into the Dictinary
→ in 3.7 version before, this(popitem) remove random method.
→ it return a tuple → that removed items.

del :-

→ it can able to total dictionary or specified item with the key value

```
a = {"name": 'guru', 'age': 21}
del a["name"]
print(a)        # {"age": 21}

del a → This will entire object.
```

clear() :-

→ it empties the total dictinary items.

```
a = {"name": 'guru', 'age': 21}
a.clear()
print(a)        # {}
```

Loop through a Dict :-

→

```
d = {"name": "guru", "age": 21}   # for keys
for i in d:
    print(i)        # {"name", "age"}
```

→

```
d = {"name": "guru", "age": 21}

for i in d.values():
    print(i)        # {"guru", 21}
```

```
d = {"name": "guru", 'age': 21}
for i,j in d.items():
    print(i,j)
```

Copy :-

copy() :-

→ This method copy a specified dictionary

```
a = {"name": "guru", "age": 21}
b = a.copy()
print(b)
```

dict :-

→ it take the keyword arguments and returns the dict

```
a = {"name": 'guru', "age": 21}
b = dict(a)
print(b)   # {"name": 'gguru', "age", 21}
```